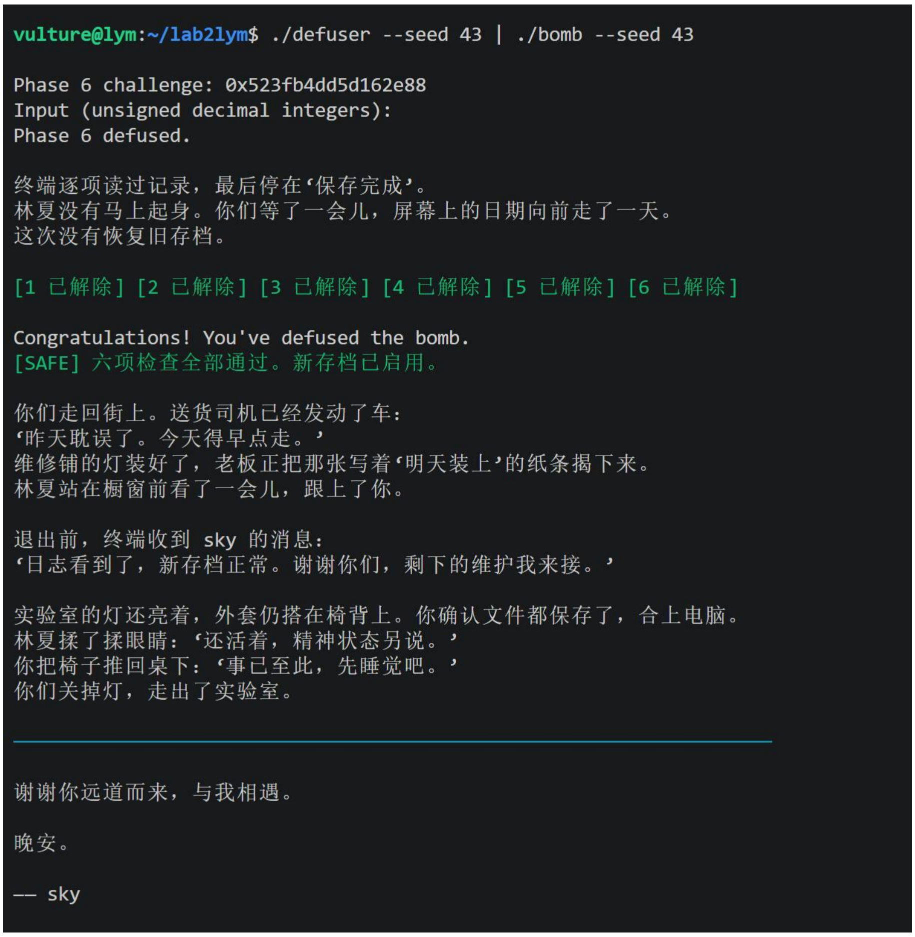
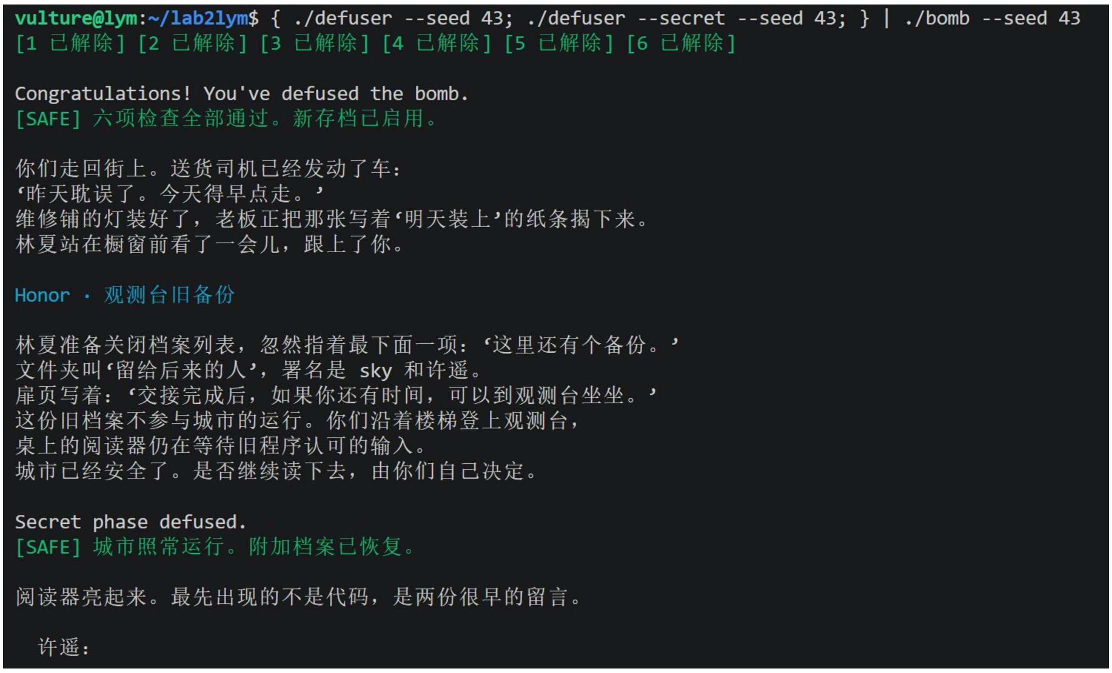
Lab2: BombLab 实验报告

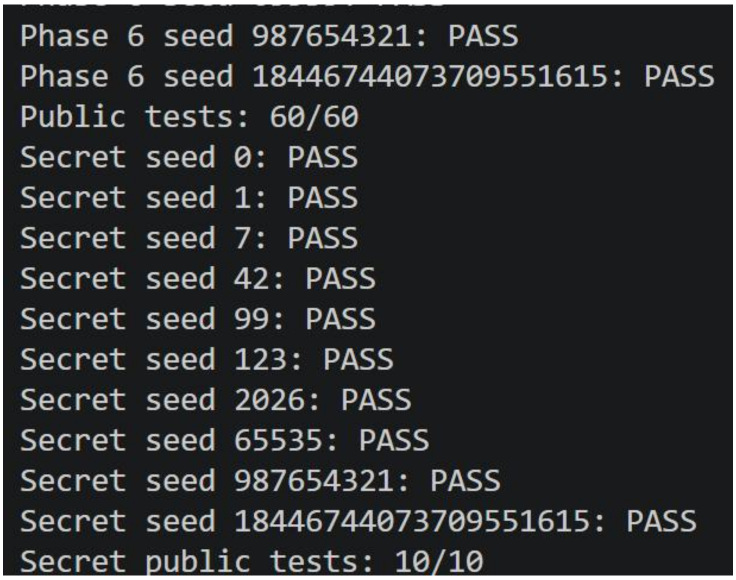
一、运行验证截图：



```
vulture@lym:~/lab2lym$ ./defuser --seed 43 | ./bomb --seed 43

Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.

终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

________________________________________________________________________


谢谢你远道而来，与我相遇。

晚安。

—— sky
```



```
vulture@lym:~/lab2lym$ { ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：
```



```
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

二、解题思路

Phase_1：

通过 valid_u32_fields(input, 1)和 sscanf 后与 1 比较，可以确定口令只有 1 个整数。p1_compute 的返回值与输入比较，可以推出答案就是 p1_compute(challenge)。

p1_compute 将 challenge 拆分为 L = low32(challenge), H = high32(challenge), S = challenge >> 16，然后调用 step0、step1、step2 三个函数分别进行计算。step2 计算(H+2L) ^ 0x2c61u，step0 计算(3*L+S)^0x1f10，step1 利用前两个结果，计算(3*r0)^(r2+0x4cab)。

Phase_2：

从 valid_u32_fields(input,6)和 sscanf 的 6 个输出槽可确定共 6 个整数。第一个整数由(challenge>>31)*2 后取低 8 位再加 0x18 得到。循环中 rcx 从 13 开始每次加 2，rdx 为循环变量 i，从 1 到 5，每轮循环+1。第 i 个整数的计算方式为：challenge >> (13+2(i-1))，取低 4 位，+i，再^3*input[i-1]。input[i-1] 通过 esi 在循环中保留并更新。

Phase_3：

通过 valid_u32_fields(input, 2)可确定共 2 个整数。解析输入的整数，input[0]是(challenge & 7)，作为 p3_dispatch 的第一个变量，即索引。p3_dispatch 的第二个变量为 x，传入各个 case 计算结果，x = (challenge>>32) ^ (challenge>>3)。

p3_dispatch 通过跳转表选择 case。跳转表索引顺序不等于源码 case 顺序，最初按 idx=i → case_i 实现导致失败。之后用 GDB 在 call p3_dispatch 前读 esi、eax，并用 x/8gx 0x405820 查看表项，确认 idx=4 实际对应 case2，以此类推获得正确索引表。input[1]是 p3_dispatch 的返回值。

Phase_4：

通过 valid_u32_fields(input, 1)可确定口令为 1 个整数。Phase_4 为 p4_walk 准备四个参数，分别是 a = challenge & 0x7f，b = 0，c = 0x7f，d = (challenge>>7) ^

(challenge>>32)。

p4_walk 在[b,c]中二分查找 a，a 命中中点后返回 (a+0x3a)^d，再沿递归返回路径每层计算值。设 mid + d = base，a < mid 时返回值为 ((5 * 内层返回值) ^ base) ^ 0xb，a > mid 时返回值为 ((5 * rec) ^ base) ^ 0x9。递归反向传递完成后得到的函数返回值即为结果。

Phase_5：

通过 valid_u32_fields(input, 2)可确定共 2 个整数。

p5_traverse 中出现了 mov 0x405b60(,%rax,8),%rdx。已知 rax = challenge & 7，共 8 种可能值；scale = 8，每次跨度为 8 字节。而读出的值放进 rdx，rdx 后面被用作地址，说明有一个 8 项、每项 8 字节的指针表，首地址为 0x405b60。

p5_traverse 中还出现了 mov (%rdx),%rdx。该指令把 rdx 指向的内存偏移 0x0 处的值读出来，赋给 rdx 自己，之后循环访问，符合沿 next 指针遍历链表的模式。因此可确定指针表每个元素指向链表。

循环遍历次数由(challenge>>3)&3 决定，范围是 0-3，因此每个链表至少需要 4 个节点。

之后 dump 链表数据，先获得所有头指针，再获得对应的结点，最后将绝对地址换算成索引。

Phase_6：

p6_dispatch 与 p3_dispatch 都用跳转表，但这里表项是函数指针，且用 call * 调用两次。第一次用 challenge & 7 选函数，参数为 (challenge>>6)^(challenge>>32)，返回值^0xd743；第二次用(challenge>>3)&7 选函数，返回值+0xc139。同样需要按 x/8gx 0x405e00 的实际表项写映射，不能假设 idx=i → op_i。

Honor：

通过在 secret_gate 入口看参数，判断出 rdi 为第七行输入字符串，并得到 rsi 和 rdx 存储的值。通过在 solve_secret 里临时打印 challenge 和 contest，和 rsi、rdx 值比对，得到 rsi 对应 challenge，rdx 对应 context。

Secret_gate 中留出了缓冲区，并通过 sub $0x24,%rax;cmp $0x7f,%rax 进行长度检查，得出第七行长度必须在 36 到 163 之间。另外还有 cmpl $0x4e45504f,-0xb0(%rbp) 指令，已知缓冲区起始为 rbp-0xd0，此处为偏移 32，0x4e45504f 小端写法对应"OPEN"，可知第七行该位置要求为 OPEN。

secret_copy 的功能是将输入原样逐字符复制到缓冲区。

secret_phase 要求 3 个 u32 数，这 3 个数为 OPEN 之后的第七行内容。通过阅读汇编和 secret_mix 的汇编可以推出计算方式。由此可以得出第七行的所有要求，从而拼接出第七行。